

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
		Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	M Vishnu vardhan	Reg. No.:	192472087
Section / Batch:	2024	Date:	04-08-2025

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Q1. Merge Sort vs Quick Sort (100 marks)

Evaluate the robustness of Merge Sort and Quick Sort under noisy or partially corrupted datasets. Introduce controlled data inconsistencies and observe behavior. Record execution time and output correctness. Interpret results and justify algorithm reliability in imperfect conditions.

Code of Conduct

I M Vishnu Vardhan 192472087_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ch.poorva teja_____

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Matrix Multiplication
100 marks

1/1 answered

Evaluate the efficiency of divide and conquer matrix multiplication by implementing it for matrices of different sizes. Compare execution time with the standard iterative approach. Analyze how recursive division impacts performance. Interpret the results and justify the advantages and limitations of divide and conquer in matrix operations.

Your Code

```
1 import java.util.Random;
2
3 public class Main {
4
5     // Standard iterative matrix multiplication
6     static int[][] multiplyIterative(int[][] A, int[][] B, int n) {
7         int[][] C = new int[n][n];
8         for (int i = 0; i < n; i++)
9             for (int j = 0; j < n; j++)
10                 for (int k = 0; k < n; k++)
11                     C[i][j] += A[i][k] * B[k][j];
12         return C;
13     }
14
15     // Divide and Conquer matrix multiplication
16     static int[][] multiplyDivideConquer(int[][] A, int[][] B, int n)
17     {
18         int[][] C = new int[n][n];
19         if (n == 1) {
20             C[0][0] = A[0][0] * B[0][0];
21             return C;
22         }
23         int newSize = n / 2;
```

Input

stdin input

Output

Size	Iterative (ms)
Divide&Conquer (ms)	
2	0.0024
0.0097	
4	0.0047
0.0597	

- 1 The code implements **standard iterative matrix multiplication** using three nested loops.
- 2 It multiplies two $n \times n$ matrices by summing products of corresponding elements.
- 3 The experiment compares this iterative method with a **divide-and-conquer** recursive approach.
- 4 Divide-and-conquer splits matrices into sub-matrices and multiplies them recursively.
- 5 For small matrices, recursion adds overhead, making it slower than iteration.
- 6 As matrix size grows, recursion can exploit parallelism and cache locality better.
- 7 Overall, divide-and-conquer is conceptually elegant but less efficient for small inputs due to recursion cost.